

Relatorio - Sistema de E-commerce em Microservicos

Disciplina: Fundamentos de Computacao Concorrente, Paralela e Distribuida (FCCPD) | Arquitetura: API Gateway + Usuarios + Produtos (2 replicas) + Pedidos | Stack: Python/Flask, comunicacao REST/HTTPS, JWT (HS256), SQLite, frontend HTML/CSS.

1. Como a comunicacao entre os microservicos foi implementada?

A comunicacao e feita via HTTPS/REST com payloads em JSON. Todos os servicos usam TLS com certificados auto-assinados emitidos por uma CA propria (generate_certs.py), de forma que cada chamada interna e criptografada e o certificado e verificado contra a CA local - sem desabilitar a verificacao de hostname. O API Gateway e o ponto de entrada unico: recebe requisicoes do cliente e as repassa (proxy) ao microservico responsavel, preservando metodo, corpo e cabecalho Authorization (JWT). Internamente os servicos tambem conversam por HTTPS/REST: o Servico de Pedidos consulta o Servico de Produtos (GET /products/<id>) para validar o item antes de gravar, e a replica primaria de Produtos chama a replica par (POST /internal/replicate) para propagar escritas. Optou-se por REST por ser simples, sem dependencias externas alem do Flask; gRPC ou filas (RabbitMQ/Kafka) seriam alternativas para maior desempenho ou desacoplamento assincrono.

2. Qual estrategia de consistencia foi adotada na replicacao? Forte ou eventual? Por que?

Foi adotada consistencia FORTE (replicacao sincrona). Quando um produto e criado, a replica primaria grava no seu banco SQLite local e so confirma sucesso (HTTP 201) ao cliente apos a replica par tambem confirmar a gravacao. Se a propagacao para a par falhar (rede ou servico fora), a escrita local e desfeita via rollback de transacao e o cliente recebe 503 - a operacao e atomica (grava nas duas ou em nenhuma). A leitura e distribuida por round-robin entre as duas replicas. A escolha pela consistencia forte garante que qualquer replica lida devolve sempre o mesmo conjunto de produtos, evitando o cliente ver um catalogo divergente a cada leitura. O custo e menor disponibilidade de escrita: se uma replica estiver fora, nao se aceita nova escrita. Consistencia eventual traria maior disponibilidade, mas permitiria leituras temporariamente divergentes - indesejavel para um catalogo de produtos.

3. O que acontece se o Servico de Pedidos cair? O restante continua funcionando?

Sim. Como os microservicos sao independentes e tem bancos SQLite separados, a queda do Servico de Pedidos NAO afeta Usuarios nem Produtos: o cliente continua podendo registrar-se, fazer login e navegar/criar produtos normalmente. O heartbeat do Gateway detecta a falha em ate ~10 segundos (2 verificacoes sem resposta no intervalo de 5s), registra a ocorrencia em log com timestamp e passa a responder 503 (Service Unavailable) apenas para as rotas /orders, deixando claro ao cliente qual recurso esta indisponivel. Quando o servico volta, o heartbeat registra a recuperacao e o roteamento e normalizado automaticamente. Isso foi validado em teste: ao derrubar a porta 5003, o Gateway logou FALHA e retornou 503; ao reiniciar, logou RECUPERACAO.

4. Como o JWT garante que um usuario comum nao consiga criar produtos?

No login, o Servico de Usuarios gera um token JWT assinado com uma chave secreta (HS256 via hmac da stdlib Python, sem dependencia externa) contendo userId, email, role (user ou admin) e exp (expiracao). O endpoint POST /products e protegido pelo middleware require_admin, que: (a) extrai e valida a assinatura do token com a mesma chave secreta - se o token foi adulterado, a assinatura nao confere e a requisicao e rejeitada com 401; (b) verifica se nao expirou; e (c) exige que o campo role seja exatamente 'admin', retornando 403 caso contrario. Como o role esta dentro do payload assinado, um usuario comum nao consegue forja-lo: alterar o role invalidaria a assinatura. Assim, mesmo possuindo um token valido de usuario, ele recebe 403 ao tentar criar produtos - comportamento confirmado em teste manual.

5. Quais limitacoes a implementacao possui em relacao a um sistema real de

producao?

- Servidor de desenvolvimento do Flask (Werkzeug), nao um WSGI de producao (gunicorn/uvicorn) atras de um balanceador de carga real.
- Replicacao apenas entre 2 nos, sem eleicao de lider, sem reconciliacao automatica apos particao de rede; quando uma replica esta fora, escritas falham.
- Certificados TLS auto-assinados por CA propria: adequados para desenvolvimento e comunicacao interna, mas em producao usaria-se certificados emitidos por uma CA publica (ex: Let's Encrypt) ou um servico de gestao de certificados (Vault, AWS ACM).
- Chave JWT e segredo de replicacao em variaveis de ambiente - em producao usar-se-ia um cofre de segredos (HashiCorp Vault, AWS Secrets Manager).
- Sem refresh token, revogacao de token, rate limiting, observabilidade distribuida (metricas/tracing com Prometheus/Jaeger) nem circuit breaker robusto (Hystrix/Resilience4j).
- Heartbeat simples com estado em memoria do Gateway, sem persistencia nem alta disponibilidade do proprio Gateway, que e um ponto unico de falha (SPOF).
- SQLite e adequado para este escopo educacional, mas nao escala horizontalmente; em producao usaria-se PostgreSQL/MySQL com replicas gerenciadas.

Apesar dessas limitacoes, o projeto cumpre os objetivos didaticos: decomposicao em microservicos independentes, persistencia em SQLite, replicacao com consistencia forte, deteccao de falha por heartbeat, seguranca com JWT e comunicacao interna via HTTPS/TLS com CA propria (item opcional implementado).